Saksham Gupta

Systems Software Engineer | Performance Engineering, JVM Internals, Concurrency

sakshamdam54321@gmail.com +91 8687-667561 Blog sakshamgupta28 0mega28

Experience

Oracle, Principal Platform Software Engineer *Bengaluru, IN Oct 2026 – Present*
Oracle, Project Lead, Senior Platform Software Engineer *Bengaluru, IN Oct 2025 – Oct 2026*

- Architected a **high-performance declarative rendering engine** (Siebel Web Component Framework) with a recursive JSON layout grammar, composable expression layer, and signal-based reactive state management using Preact internals, handling large DOM trees efficiently with strong validation and debuggability.
- Built a **signal store** that re-binds listeners when components detach and reattach, covering for a missing unbind API in OpenUI and simplifying state management across the framework.
- Designed a **DSL-driven integration orchestration framework** (Siebel Open Integration – Phase 2) with external DSL + internal DSL model for composing multi-step workflows across REST, eventing, and data-access layers with declarative transitions, conditions, and state management. Learned how Kafka works to design the eventing configuration.
- Established **CI/CD pipelines** and built an AI-assisted unit testing framework, authoring **100+ test cases** to enforce code quality standards across the team.
- Mentored engineers across 3 stacks (Java, JavaScript, Python), led architecture reviews, and wrote design docs before implementation.

Oracle, Senior Software Engineer *Bengaluru, IN Sept 2024 – Sept 2025*

- Designed and built a **RAG (Retrieval-Augmented Generation) system** over Siebel Bookshelf with a vector database, LangChain, and LLMs, leading two engineers. It later grew into a full AI developer experience for Siebel.
- Led migration of **Siebel Open Integration from Java 17 to Java 21** with Virtual Threads and Helidon 4 upgrade, resolving critical synchronous/asynchronous proxy bugs and authoring a new error-handling module.
- Tracked down a **memory leak** in the GraalVM JavaScript runtime by attaching Chrome DevTools and diffing heap snapshots across interactions. Traced the growth to business-component field references that were never released.
- Automated third-party dependency management by implementing a **declarative package.json-based system**, eliminating manual downloads. Awarded the **"Continuous Innovation" Award**.

Oracle, Software Engineer *Bengaluru, IN Aug 2022 – Aug 2024*

- Led **Siebel Open Integration from proof-of-concept to beta in 12 months**, building the entire server-side execution layer from scratch.
- **Reduced CPU usage by 60% and API latency by 75%** in the GraalVM polyglot service. Profiled with flamegraphs and CPU samplers and removed redundant object cloning on hot paths, keeping copies only where callers needed protection from parent-object mutation.
- Engineered a **polyglot runtime** using GraalVM JavaScript contexts embedded in a Java Helidon service, enabling server-side execution of client-side business logic for many concurrent users.
- Ran the embedded JavaScript under GraalVM's **UNTRUSTED sandbox policy**, restricting guest code's access to the host JVM. Reported a missing regex capability in that policy, which the GraalVM team then added.
- Made the **Java–JavaScript bridge non-blocking**: JavaScript calls return Promises that Java adapts to CompletableFutures for chaining, so request threads never block.
- Designed a **session locator** that caches initialized Siebel sessions and assigns the best available one to each request.
- Cut **first-request latency** by warming up the JIT at startup: the hot path runs repeatedly with stub data so GraalVM has profiles to compile the JavaScript before real traffic arrives.
- Built **automated REST API code generation** using OpenAPI specifications and GraalVM polyglot capabilities, exposing Siebel domain objects as typed resource hierarchies with a developer-friendly DSL.
- Diagnosed a **self-induced thread-pool deadlock** in the Siebel Application Interface team's Helidon 3 evaluation, where CompletableFuture tasks submitted to the ForkJoin pool were joined from that same pool's threads, and

helped the team fix it.

Skills

Languages: Java, C++, JavaScript, Bash, Python, Rust, Haskell

Systems & Runtimes: GraalVM Polyglot, Helidon, Virtual Threads, performance profiling (flamegraphs, CPU samplers, VisualVM), concurrency & multithreading, REST API design, code generation (OpenAPI)

Architecture & Design: DSL design (external/internal), workflow orchestration, framework architecture, reactive programming, recursive descent parsing

AI/ML: RAG pipelines, LangChain, vector search / embeddings

Education

Indian Institute of Technology, Roorkee BTech, Computer Science *2018 – 2022*

- **GPA**: 8.42 (Degree) | **JEE (Advanced)**: AIR 353 (Result)

Awards

- **2x "Continuous Innovation" Award** (FY24 Q4, FY25 Q3) — recognized for building a greenfield API platform with GraalVM polyglot and automating dependency management.
- **"Agency in Action" Award** (FY25 Q3) — resolved 25+ compliance and accessibility issues, directly supporting critical release deliveries.
- **4 consecutive "Outstanding" performance ratings** (FY23–FY26), and three promotions in four years.
- **Top 6 in Oracle Global GenAI Hackathon** — 101 teams worldwide.

Projects

- **OS From Scratch** (C, x86 Assembly) (GitHub) — 32-bit UNIX clone kernel with custom bootloader, interrupt handling, virtual memory manager, VGA driver, memory allocator, and shell. Multi-boot compliant; runs on QEMU.
- **CPU Simulator — CENOS** (C++) (GitHub) — Simulates Out-of-Order execution pipeline with Scoreboarding, branch prediction, and a 2-pass assembler on a RISC-V-inspired ISA.
- **HTTP Proxy Server** (C++) (GitHub) — Minimal HTTP proxy using RAII for socket safety and custom URL parsing; forwards client requests to remote hosts.
- **SAVY — Synchronized Video Player** (JavaScript) (GitHub) — Real-time video synchronization across clients using Node.js, Socket.io, and WebRTC audio chat.
- **CacheCraft** (Java) (GitHub) — Implementation of various caching strategies and design patterns (LRU, LFU, write-through, write-back).
- **JSON Parser** (Java) (GitHub) — Recursive descent parser for JSON objects, arrays, strings, numbers, booleans, and null values.
- **Math Expression Evaluator** (Java) (GitHub) — Expression parser & evaluator using functional parser-combinator techniques.

Technical Writing

- **Blog:** 0mega28.github.io — Deep dives into Java internals, distributed systems, and systems programming.
- Why Distributed Systems Can't Trust the Clock — Lamport's 1978 paper: how the happens-before relation and logical clocks order events by causality instead of physical time.
- Capturing Distributed State: The Chandy-Lamport Algorithm — How to snapshot a running distributed system without pausing it, using consistent cuts and marker messages.
- A Systems-First Deep Dive into the Spark Paper — Dissecting the 2012 RDD paper: how Spark's functional, lineage-based architecture solved fault tolerance without replication.
- The JVM Has Never Heard of Your List<String> — Technical deep dive into Java generics type erasure and bytecode-level implications.